

Work Integrated Learning Programmes Division

M. Tech. in Artificial Intelligence & Machine Learning

1st Semester 2026-27

Assignment I

AIMLZC416 — Mathematical Foundations for Machine Learning

2026aj05009 - Saurabh Tayal

Q1) Finding solutions of linear systems

1. Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing $\square$ REF, and $\square$ RREF without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b . Note that this part should be explicitly there in the code and give a comment line on the same. (1 mark + 1 mark)

```
In [1]: import numpy as np

# Q1(1): Define reusable functions to calculate REF and RREF of [A | b].
# A has m rows and n coefficient columns, and b has m rows and one column.
# Appending b to A creates the augmented matrix [A | b] with shape m x (n + 1).
# The row operations below are written explicitly to show the elimination logic.

def matrix_multiply(first_matrix, second_matrix):
    # If the first matrix is p x q and the second is q x r, their product is p x r
    first_rows = len(first_matrix)
    first_columns = len(first_matrix[0])
    second_rows = len(second_matrix)
    second_columns = len(second_matrix[0])

    if first_columns != second_rows:
        raise ValueError("The matrices cannot be multiplied: inner dimensions do not match")

    product = np.zeros((first_rows, second_columns))
    for i in range(first_rows):
        for j in range(second_columns):
            total = 0.0
            for k in range(first_columns):
                total += first_matrix[i, k] * second_matrix[k, j]
            product[i, j] = total
    return product

def transpose_matrix(matrix):
    # The transpose changes rows into columns. The original entry at (i, j) is now at (j, i)
    rows = len(matrix)
    columns = len(matrix[0])
    transposed = np.zeros((columns, rows))
```

```

for i in range(rows):
    for j in range(columns):
        transposed[j, i] = matrix[i, j]
return transposed

def ref_form(augmented_matrix, tolerance=1e-12):
    # Use a floating-point copy so row operations do not modify the input.
    ref_matrix = augmented_matrix.copy().astype(float)
    number_of_rows, number_of_columns = ref_matrix.shape
    coefficient_columns = number_of_columns - 1
    pivot_row = 0

    # Scan coefficient columns from left to right. 'pivot_row' means the next row
    for column in range(coefficient_columns):
        # If all rows already contain pivots, no further elimination is needed.
        if pivot_row == number_of_rows:
            break

        # Search the current row and all rows below it for a non-zero entry.
        # Treat very small floating-point values as zero using the tolerance.
        candidate_row = pivot_row
        while (
            candidate_row < number_of_rows
            and abs(ref_matrix[candidate_row, column]) < tolerance
        ):
            candidate_row += 1

        # If no usable entry is found, the column has no pivot. Skip it instead
        if candidate_row == number_of_rows:
            continue

        # Move the usable row into the next pivot position. Row swaps preserve t
        ref_matrix[[pivot_row, candidate_row]] = ref_matrix[[candidate_row, piv

        # The pivot is non-zero because of the search above, so division is safe
        # Scale the complete row so that its pivot entry becomes exactly 1.
        ref_matrix[pivot_row] = ref_matrix[pivot_row] / ref_matrix[pivot_row, co

        # Subtract a multiple of the pivot row from each lower row. This makes e
        for next_row in range(pivot_row + 1, number_of_rows):
            multiplier = ref_matrix[next_row, column]
            ref_matrix[next_row] = (
                ref_matrix[next_row] - multiplier * ref_matrix[pivot_row]
            )

        # A pivot was successfully created, so the next pivot belongs in the nex
        pivot_row += 1

    return ref_matrix

def rref_form(augmented_matrix, tolerance=1e-12):
    # REF has zeros below each pivot. Begin with REF and remove entries above ea
    rref_matrix = ref_form(augmented_matrix, tolerance)
    number_of_rows, number_of_columns = rref_matrix.shape
    coefficient_columns = number_of_columns - 1

    # Process rows from bottom to top. This ensures that eliminating an entry ab
    for pivot_row in range(number_of_rows - 1, -1, -1):

```

```

    # The first non-zero coefficient in this row identifies its pivot column
    pivot_column = 0
    while (
        pivot_column < coefficient_columns
        and abs(rref_matrix[pivot_row, pivot_column]) < tolerance
    ):
        pivot_column += 1

    # If no coefficient is non-zero, this is an all-zero row with no pivot.
    if pivot_column == coefficient_columns:
        continue

    # The pivot was normalized to 1 during REF. Eliminate its entries from e
    for previous_row in range(pivot_row):
        multiplier = rref_matrix[previous_row, pivot_column]
        rref_matrix[previous_row] = (
            rref_matrix[previous_row] - multiplier * rref_matrix[pivot_row]
        )

    return rref_matrix

while True:
    print("Enter the size of A = m x n, where m < n")
    m = int(input("m = "))
    n = int(input("n = "))
    if m > 0 and n > 0 and m < n:
        break
    print("Please enter positive values with m < n.")

# Generate reproducible random decimal entries as required by the assignment.
np.random.seed(2026)
A = np.random.uniform(0.1, 9.9, size=(m, n))
# Generate an independent random right-hand-side vector b.
b = np.random.uniform(0.1, 9.9, size=(m, 1))
augmented = np.hstack((A, b))

print("\nMatrix A =")
print(A)
print("\nVector b =")
print(b)
print("\nAugmented matrix [A | b] =")
print(augmented)

# Compute and display REF and RREF. The functions are reused in Q1.2 and Q1.3.
ref = ref_form(augmented)
rref = rref_form(augmented)

print("\nREF =")
print(ref)
print("\nRREF =")
print(rref)

```

Enter the size of $A = m \times n$, where $m < n$

Matrix $A =$

```
[[2.24958722 4.14751502 9.67102769]
 [0.97121041 4.79704062 9.77799484]]
```

Vector $b =$

```
[[2.04178267]
 [9.03480878]]
```

Augmented matrix $[A \mid b] =$

```
[[2.24958722 4.14751502 9.67102769 2.04178267]
 [0.97121041 4.79704062 9.77799484 9.03480878]]
```

REF =

```
[[1.          1.84367824 4.29902321 0.90762548]
 [0.          1.          1.8635784  2.7119485  ]]
```

RREF =

```
[[ 1.          0.          0.86318425 -4.09233497]
 [ 0.          1.          1.8635784  2.7119485  ]]
```

2. Write a Python code to identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax = 0$. (1 mark)

```
In [2]: # Q1(2): Continue with A, b, rref_form, and matrix_multiply from part 1.1.
# This cell also defines the consistency check needed before solving  $Ax = b$ .
# The RREF identifies pivot variables and free variables.

def has_inconsistent_row(rref_matrix, coefficient_columns, tolerance=1e-12):
    # In the RREF of  $[A \mid b]$ , a row of the form  $[0 \ 0 \ \dots \ 0 \mid c]$ , where  $c$  is non-
    # This contradiction means that the system  $Ax = b$  has no solution.
    rows = len(rref_matrix)
    for row in range(rows):
        coefficients_are_zero = True

        # Check whether every coefficient in this row is effectively zero.
        # The tolerance handles tiny round-off values produced by floating-point
        for column in range(coefficient_columns):
            if abs(rref_matrix[row, column]) >= tolerance:
                coefficients_are_zero = False
                break

        # If all coefficients are zero but the augmented value is non-zero, the
        if (
            coefficients_are_zero
            and abs(rref_matrix[row, coefficient_columns]) >= tolerance
        ):
            return True

    # No contradictory row was found, so the system is consistent.
    return False

# Recalculate RREF from the same augmented matrix created in part 1.1.
rref = rref_form(augmented)

# A consistency test must happen before extracting solutions. If the system is i
```

```

if has_inconsistent_row(rref, n):
    print("The system is inconsistent and has no solution.")
    raise SystemExit("Program stopped because no solution exists.")

# Identify pivot columns by scanning each RREF row from left to right.
# The first non-zero coefficient in a non-zero row is that row's pivot.
pivot_columns = []
for row in range(m):
    for column in range(n):
        if abs(rref[row, column]) > 1e-12:
            pivot_columns.append(column)
            break

# Every coefficient column that is not a pivot column represents a free variable
# These variables can be assigned arbitrary values.
non_pivot_columns = []
for column in range(n):
    if column not in pivot_columns:
        non_pivot_columns.append(column)

print("Pivot columns (0-based):", pivot_columns)
print("Non-pivot columns (0-based):", non_pivot_columns)

# Choose zero for every free variable. Reading the augmented column then gives a
particular_solution = np.zeros(n)
for row, pivot_column in enumerate(pivot_columns):
    # 'row' identifies the RREF equation, while 'pivot_column' identifies the augmented column
    particular_solution[pivot_column] = rref[row, n]

print("\nParticular solution x_p =")
print(particular_solution)
print("Check A x_p =")
# Reshape x_p into an n x 1 column vector because A has shape m x n.
# The manual multiplication returns an m x 1 vector, which is flattened for display
print(matrix_multiply(A, particular_solution.reshape(n, 1)).reshape(m))
print("Check b =")
print(b.reshape(m))

# Construct homogeneous solutions for Ax = 0. Process one free variable at a time
# set the selected free variable to 1 and all other free variables to 0.
null_space_vectors = []
for free_column in non_pivot_columns:
    vector = np.zeros(n)
    vector[free_column] = 1

    # For each RREF equation, solve the pivot variable in terms of the selected free variable
    for row, pivot_column in enumerate(pivot_columns):
        vector[pivot_column] = -rref[row, free_column]
    null_space_vectors.append(vector)

print("\nVectors for solutions of A x = 0:")
for number, vector in enumerate(null_space_vectors, start=1):
    print("v" + str(number) + " =", vector)
    print("A v" + str(number) + " =")
    # Each result should be the zero vector, apart from insignificant floating-point errors
    print(matrix_multiply(A, vector.reshape(n, 1)).reshape(m))

# The complete solution of Ax = b is one particular solution plus any linear combination of null space vectors
print("\nGeneral solution:")
print("x = x_p + c1*v1 + c2*v2 + ...")

```

Pivot columns (0-based): [0, 1]
 Non-pivot columns (0-based): [2]

Particular solution $x_p =$
 $[-4.09233497 \quad 2.7119485 \quad 0.]$
 Check $A x_p =$
 $[2.04178267 \quad 9.03480878]$
 Check $b =$
 $[2.04178267 \quad 9.03480878]$

Vectors for solutions of $A x = 0$:
 $v_1 = [-0.86318425 \quad -1.8635784 \quad 1.]$
 $A v_1 =$
 $[-1.77635684e-15 \quad 0.00000000e+00]$

General solution:
 $x = x_p + c_1 v_1 + c_2 v_2 + \dots$

3. Consider a random 5×7 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution. ($1/4 \times 8 = 2$ marks)

```
In [3]: # Q1(3): Continue using ref_form, rref_form, matrix_multiply, and has_inconsiste
# This cell applies the complete solution process to the required 5 x 7 example.
# A 5 x 7 system has 5 equations and 7 variables, so a full-row-rank example nor

np.random.seed(2026)
A_5x7 = np.random.uniform(0.1, 9.9, size=(5, 7))
# Generate b independently, as required. The solution will be obtained later
# from the RREF; it is not supplied in advance.
b_5x7 = np.random.uniform(0.1, 9.9, size=(5, 1))
augmented_5x7 = np.hstack((A_5x7, b_5x7))

# Reuse the functions from Q1.1. Row reduction transforms the augmented system b
ref_5x7 = ref_form(augmented_5x7)
rref_5x7 = rref_form(augmented_5x7)

print("A =")
print(A_5x7)
print("\nb =")
print(b_5x7.reshape(5))
print("\nREF of [A | b] =")
print(ref_5x7)
print("\nRREF of [A | b] =")
print(rref_5x7)

# Before solving, test for a row [0 0 ... 0 | c] with c non-zero.
# Such a row represents the contradiction 0 = c and proves that no solution exists
if has_inconsistent_row(rref_5x7, 7):
    print("\nThe 5 x 7 system is inconsistent and has no solution.")
    raise SystemExit("Program stopped because no solution exists.")

# Scan only the seven coefficient columns. The final column is b and must not be
pivot_columns_5x7 = []
for row in range(5):
    for column in range(7):
        if abs(rref_5x7[row, column]) > 1e-12:
            pivot_columns_5x7.append(column)
```

```

        break

# Columns without pivots correspond to free variables. Their values determine the
non_pivot_columns_5x7 = [
    column for column in range(7) if column not in pivot_columns_5x7
]

print("\nPivot columns:", pivot_columns_5x7)
print("Non-pivot columns:", non_pivot_columns_5x7)

# Set all free variables to zero. The augmented value in each pivot row then gives
particular_solution_5x7 = np.zeros(7)
for row, pivot_column in enumerate(pivot_columns_5x7):
    particular_solution_5x7[pivot_column] = rref_5x7[row, 7]

print("\nParticular solution x_p =")
print(particular_solution_5x7)
print("Check A x_p =")
print(matrix_multiply(A_5x7, particular_solution_5x7.reshape(7, 1)).reshape(5))
print("Check b =")
print(b_5x7.reshape(5))

# Build one vector for the null space for each free variable.
# For each vector, one free variable is set to 1 and the other free variables are
null_space_vectors_5x7 = []
for free_column in non_pivot_columns_5x7:
    vector = np.zeros(7)
    vector[free_column] = 1
    for row, pivot_column in enumerate(pivot_columns_5x7):
        vector[pivot_column] = -rref_5x7[row, free_column]
    null_space_vectors_5x7.append(vector)

print("\nSolutions to A x = 0:")
for number, vector in enumerate(null_space_vectors_5x7, start=1):
    print("v" + str(number) + " =", vector)
    print("A v" + str(number) + " =")
    # These products should be zero, with tiny numerical round-off values allowed
    print(matrix_multiply(A_5x7, vector.reshape(7, 1)).reshape(5))

# Since this example has two free variables, every solution is described by two
print("\nGeneral solution:")
print("x = x_p + c1*v1 + c2*v2")
print("The checks above verify A x_p = b and A v_i = 0.")

```

A =

```
[[2.24958722 4.14751502 9.67102769 0.97121041 4.79704062 9.77799484
 2.04178267]
[9.03480878 5.57085113 7.77564879 3.06427058 3.99354974 3.00047968
 8.86206692]
[0.57211601 4.4616629 5.60656011 3.66440338 6.90603218 9.1490072
 0.39981206]
[1.2897733 2.37268242 6.40039011 2.41214929 9.57528454 1.5618379
 0.55958511]
[4.71042124 6.78925934 8.71795496 6.79261965 5.19880891 4.67665447
 2.62252588]]
```

b =

```
[5.07867715 5.72980397 4.90339048 2.130364 4.51865889]
```

REF of [A | b] =

```
[[ 1. 1.84367824 4.29902321 0.43172828 2.13240926 4.34657289
 0.90762548 2.25760402]
[-0. 1. 2.80209282 0.07543564 1.37757251 3.27156512
 -0.0596986 1.32298834]
[-0. -0. 1. -0.49386493 -0.15514792 0.70062275
 -0.01311544 0.1399308 ]
[ 0. 0. 0. 1. 3.04839611 -2.02878805
 -0.26249639 -0.39215322]
[-0. -0. -0. -0. 1. 0.17396077
 0.15579005 0.23018352]]
```

RREF of [A | b] =

```
[[ 1. 0. 0. 0. 0. -1.33034018
 0.99075339 -0.08380277]
[ 0. 1. 0. 0. 0. 4.72754021
 0.77080223 2.10996141]
[ 0. 0. 1. 0. 0. -0.5362321
 -0.353124 -0.36456778]
[ 0. 0. 0. 1. 0. -2.55908938
 -0.73740619 -1.09384376]
[-0. -0. -0. -0. 1. 0.17396077
 0.15579005 0.23018352]]
```

Pivot columns: [0, 1, 2, 3, 4]

Non-pivot columns: [5, 6]

Particular solution x_p =

```
[-0.08380277 2.10996141 -0.36456778 -1.09384376 0.23018352 0.
 0.]
```

Check A x_p =

```
[5.07867715 5.72980397 4.90339048 2.130364 4.51865889]
```

Check b =

```
[5.07867715 5.72980397 4.90339048 2.130364 4.51865889]
```

Solutions to $Ax = 0$:

```
v1 = [ 1.33034018 -4.72754021 0.5362321 2.55908938 -0.17396077 1.
 0.]
```

A v1 =

```
[-5.32907052e-15 -2.26485497e-14 -1.77635684e-15 -4.44089210e-16
 -7.99360578e-15]
```

```
v2 = [-0.99075339 -0.77080223 0.353124 0.73740619 -0.15579005 0.
 1.]
```

A v2 =

```
[-4.44089210e-16 1.77635684e-15 -3.33066907e-16 3.33066907e-16
```

-1.77635684e-15]

General solution:

$x = x_p + c_1 v_1 + c_2 v_2$

The checks above verify $A x_p = b$ and $A v_i = 0$.

Q2) Consider a dataset $X \in \mathbb{R}^{500 \times 6}$ constructed as follows: the first four features f_1, f_2, f_3, f_4 are generated as random features sampled from a standard normal distribution (read about this). The fifth and sixth features are defined by the relations $f_5 = 2f_1 + 3f_2$, $f_6 = f_3 - 2f_4$. Perform the following tasks in sequence:

1. Write a Python code to generate the dataset $X = [f_1, f_2, f_3, f_4, f_5, f_6]$. [0.5]

```
In [4]: import numpy as np

# Q2(1): Create X = [f1, f2, f3, f4, f5, f6].
# Each row of X is one data point and each column is one feature.
# The first four features are independent standard-normal random variables.
# The last two features are deliberately dependent on the first four.

# The seed makes the random dataset reproducible. Re-running this cell gives exact results.
np.random.seed(2026)
n = 500

# randn draws values from N(0, 1), the standard normal distribution.
f1 = np.random.randn(n)
f2 = np.random.randn(n)
f3 = np.random.randn(n)
f4 = np.random.randn(n)

# These two columns are linear combinations of earlier columns. Consequently, the dataset is not full rank.
f5 = 2 * f1 + 3 * f2
f6 = f3 - 2 * f4

# column_stack places the six length-n arrays side by side. Thus X has 500 rows
X = np.column_stack((f1, f2, f3, f4, f5, f6))

print("Shape of X:", X.shape)
print("First five rows of X:")
print(X[:5])
```

Shape of X: (500, 6)

First five rows of X:

```
[[-0.43171852 -0.70255252 -0.10353308  0.87732991 -2.9710946  -1.85819289]
 [-1.39287397  1.50152579 -0.73937348  1.6569189   1.71882945 -4.05321129]
 [ 0.31157067 -0.60182461  1.21508673  1.27213227 -1.1823325  -1.32917782]
 [-0.01323488  1.44440069  1.47784444  1.3170826   4.3067323  -1.15632076]
 [ 1.44970773 -0.29466117 -0.19591321  0.71605898  2.01543195 -1.62803116]]
```

2. Write a Python code which computes the rank of X and display the output for the dataset X generated in step 1. [0.5]

```
In [5]: import numpy as np

# Q2(2): Find the rank of X without using a built-in rank function.
# Rank is the number of pivot columns obtained after row reduction.
# The first four features are independent, while f5 and f6 are combinations of the first four.
```

```

# Make a floating-point copy so row operations do not change the original X.
reduced_X = X.copy().astype(float)
number_of_rows, number_of_columns = reduced_X.shape
rank_of_X = 0
tolerance = 1e-12

# Process each column from left to right and search for a pivot.
for column in range(number_of_columns):
    # Stop if every row already has a pivot.
    if rank_of_X == number_of_rows:
        break

    # Search the current row and the rows below it for a non-zero entry.
    pivot_row = rank_of_X
    while pivot_row < number_of_rows and abs(reduced_X[pivot_row, column]) < tol:
        pivot_row += 1

    # If the entire remaining part of this column is zero, it has no pivot.
    # Skip it instead of dividing by zero and continue with the next column.
    if pivot_row == number_of_rows:
        continue

    # Move the non-zero pivot row into the next available pivot position.
    reduced_X[[rank_of_X, pivot_row]] = reduced_X[[pivot_row, rank_of_X]]

    # The pivot is non-zero because of the check above, so this division is safe
    # Normalize the pivot row so that its pivot becomes 1.
    reduced_X[rank_of_X] = reduced_X[rank_of_X] / reduced_X[rank_of_X, column]

    # Eliminate entries below the pivot. This creates row-echelon form.
    for next_row in range(rank_of_X + 1, number_of_rows):
        multiplier = reduced_X[next_row, column]
        reduced_X[next_row] = reduced_X[next_row] - multiplier * reduced_X[rank_of_X]

    # One successful pivot represents one linearly independent direction.
    rank_of_X += 1
# print("\nRow-reduced form of X:")
# print(reduced_X)
print("Rank of X:", rank_of_X)

```

Rank of X: 4

3. Numerical Experiment with the Power Method Read about the power method for finding the dominant eigenvalue and its corresponding eigenvector and perform the following tasks.

(a) Write a Python code to compute the covariance matrix [0.5]

$C = (1/n)X^T X$ where n is number of data points

```

In [6]: # Q2(3a): Compute  $C = (1/n) X^T X$  without using built-in transpose or matrix-mul
# X has n rows and 6 feature columns. Therefore  $X^T$  has shape 6 x n, and  $X^T X$  h

def transpose_matrix(matrix):
    # The transpose changes rows into columns. For every original element matrix
    rows = len(matrix)
    columns = len(matrix[0])

```

```

transposed = np.zeros((columns, rows))
for i in range(rows):
    for j in range(columns):
        transposed[j, i] = matrix[i, j]
return transposed

def matrix_multiply(first_matrix, second_matrix):
    # Use the definition of matrix multiplication. Each output entry is the sum
    first_rows = len(first_matrix)
    first_columns = len(first_matrix[0])
    second_rows = len(second_matrix)
    second_columns = len(second_matrix[0])

    if first_columns != second_rows:
        raise ValueError("The matrices cannot be multiplied: inner dimensions di

    product = np.zeros((first_rows, second_columns))
    for i in range(first_rows):
        for j in range(second_columns):
            for k in range(first_columns):
                product[i, j] += first_matrix[i, k] * second_matrix[k, j]
    return product

n = X.shape[0]
transposed_X = transpose_matrix(X)
C = matrix_multiply(transposed_X, X) / n

print("n =", n)
print("Shape of X^T =", transposed_X.shape)
print("Covariance matrix C =")
print(C)

```

```

n = 500
Shape of X^T = (6, 500)
Covariance matrix C =
[[ 9.99410950e-01 -1.60051587e-02  1.89588472e-02  9.93510963e-03
   1.95080642e+00 -9.11372085e-04]
 [-1.60051587e-02  1.11925929e+00 -1.33242656e-01  6.06364469e-02
   3.32576755e+00 -2.54515550e-01]
 [ 1.89588472e-02 -1.33242656e-01  1.14325526e+00 -3.01520021e-02
  -3.61810275e-01  1.20355926e+00]
 [ 9.93510963e-03  6.06364469e-02 -3.01520021e-02  9.94448097e-01
   2.01779560e-01 -2.01904820e+00]
 [ 1.95080642e+00  3.32576755e+00 -3.61810275e-01  2.01779560e-01
   1.38789155e+01 -7.65369395e-01]
 [-9.11372085e-04 -2.54515550e-01  1.20355926e+00 -2.01904820e+00
  -7.65369395e-01  5.24165565e+00]]

```

(b) Implement the Power Method in Python to approximate the largest eigenvalue λ_1 and its corresponding eigenvector v_1 of C . Show the code and the outputs. [1]

```

In [7]: # Q2(3b): Use the power method to approximate the largest eigenvalue.
        # matrix_multiply and transpose_matrix were defined in Q2(3a).
        # Repeated multiplication emphasizes the eigenvector direction associated with t

def power_method(matrix, tolerance=1e-7, maximum_iterations=10000):
    # Store the starting vector as a column vector so all products use the same

```

```

vector = np.ones((matrix.shape[0], 1))
vector = vector / np.linalg.norm(vector)
old_eigenvalue = 0

for iteration in range(1, maximum_iterations + 1):
    # Multiply by the matrix, then normalize to prevent overflow or underflow
    new_vector = matrix_multiply(matrix, vector)
    vector = new_vector / np.linalg.norm(new_vector)

    # For a unit vector,  $v^T C v$  is the Rayleigh quotient and estimates  $\lambda$ 
    transpose_vector = transpose_matrix(vector)
    matrix_vector = matrix_multiply(matrix, vector)
    rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
    eigenvalue = rayleigh_quotient[0, 0]

    # Stop when consecutive eigenvalue estimates differ by less than tolerance
    if abs(eigenvalue - old_eigenvalue) < tolerance:
        return eigenvalue, vector.reshape(matrix.shape[0]), iteration
    old_eigenvalue = eigenvalue

return eigenvalue, vector.reshape(matrix.shape[0]), maximum_iterations

lambda_1, v1, iterations_1 = power_method(C)

print("Largest eigenvalue:", lambda_1)
print("Corresponding eigenvector:")
print(v1)
print("Number of iterations:", iterations_1)

```

Largest eigenvalue: 15.042254426425169

Corresponding eigenvector:

[0.13287204 0.23092481 -0.03490847 0.02798583 0.95851852 -0.09088013]

Number of iterations: 11

(c) Write a Python code to obtain the next largest eigenvalue λ_2 and its corresponding eigenvector v_2 by applying power method on $C - v_1 v_1^T C$. Having found out $v_1, v_2, \dots, v_{k-1}$, one can find λ_k and its corresponding eigenvector v_k by applying power method on $C - \sum_{j=1}^{k-1} v_j v_j^T C$. Give the code and also display the obtained eigenvalues and the corresponding eigenvectors. [1.5]

```

In [8]: # Q2(3c): Remove the first eigenvector and apply the power method again.
# matrix_multiply and transpose_matrix were defined in Q2(3a).
# Since v1 is a unit eigenvector,  $v_1 v_1^T$  projects onto its direction.
# The assignment's expression  $C - v_1 v_1^T C$  removes that direction.

def power_method(matrix, tolerance=1e-7, maximum_iterations=10000):
    # Use a column vector so the manual multiplication routine can be used.
    vector = np.ones((matrix.shape[0], 1))
    vector = vector / np.linalg.norm(vector)
    old_eigenvalue = 0

    for iteration in range(1, maximum_iterations + 1):
        new_vector = matrix_multiply(matrix, vector)
        vector = new_vector / np.linalg.norm(new_vector)

        # Compute the Rayleigh quotient  $v^T M v$  using explicit products.
        transpose_vector = transpose_matrix(vector)

```

```

matrix_vector = matrix_multiply(matrix, vector)
rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
eigenvalue = rayleigh_quotient[0, 0]

if abs(eigenvalue - old_eigenvalue) < tolerance:
    return eigenvalue, vector.reshape(matrix.shape[0]), iteration
old_eigenvalue = eigenvalue

return eigenvalue, vector.reshape(matrix.shape[0]), maximum_iterations

# Convert v1 to a column vector before forming v1 v1^T.
v1_column = v1.reshape(C.shape[0], 1)
v1_transpose = transpose_matrix(v1_column)
projection_times_C = matrix_multiply(matrix_multiply(v1_column, v1_transpose), C)
deflated_matrix = C - projection_times_C
lambda_2, v2, iterations_2 = power_method(deflated_matrix)

print("Second eigenvalue:", lambda_2)
print("Corresponding eigenvector:")
print(v2)
print("Number of iterations:", iterations_2)

```

Second eigenvalue: 6.212107735364808

Corresponding eigenvector:

[0.03619323 0.00816969 0.21055059 -0.3484544 0.09689553 0.90745939]

Number of iterations: 7

(d) Find all the eigenvalues and eigenvectors using Python function and compare with the obtained result in (c). [0.5]

In [9]: `import numpy as np`

```

# Q2(3d): Find all eigenvalues and eigenvectors using NumPy.
# This gives us a reference for checking the power-method answers.
# eigh is used because C is real and symmetric; it returns real eigenvalues and
eigenvalues, eigenvectors = np.linalg.eigh(C)

# eigh returns eigenvalues in ascending order. Reverse the index order so eigenv
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

print("Eigenvalues from NumPy:")
print(eigenvalues)
print("\nEigenvectors from NumPy (columns):")
print(eigenvectors)
print("\nComparison:")
print("Power method lambda_1:", lambda_1)
print("NumPy lambda_1:", eigenvalues[0])
print("Power method lambda_2:", lambda_2)
print("NumPy lambda_2:", eigenvalues[1])

```

Eigenvalues from NumPy:

```
[1.50422544e+01 6.21210772e+00 1.12396187e+00 9.98620729e-01
 3.77522452e-15 5.75461957e-16]
```

Eigenvectors from NumPy (columns):

```
[[-0.13287044  0.03619598 -0.5000847  0.6672603  0.53448354  0.00645224]
 [-0.23092445  0.00818196  0.32657507 -0.44395781  0.80172531  0.00967836]
 [ 0.03491781  0.210542  -0.7080928  -0.53515488 -0.00492798  0.40821855]
 [-0.02800128 -0.34845685 -0.37398023 -0.26695668  0.00985596 -0.81643709]
 [-0.95851422  0.09693785 -0.02044418  0.00264718 -0.26724177 -0.00322612]
 [ 0.09092036  0.9074557  0.03986767 -0.00124151  0.00492798 -0.40821855]]
```

Comparison:

Power method lambda_1: 15.042254426425169

NumPy lambda_1: 15.042254443776912

Power method lambda_2: 6.212107735364808

NumPy lambda_2: 6.212107718406861

(e) Compare the number of iterations required by the power method to reach $1e-7$ accuracy.

```
In [10]: # Q2(3e): Count iterations needed for an eigenvalue error below 1e-7.
# matrix_multiply and transpose_matrix were defined in Q2(3a), so this cell also

def count_iterations(matrix, actual_eigenvalue, tolerance=1e-7):
    # Use the same normalized column-vector starting point as the power method.
    vector = np.ones((matrix.shape[0], 1))
    vector = vector / np.linalg.norm(vector)

    for iteration in range(1, 10000):
        # One power-method step: multiply by the matrix and normalize.
        new_vector = matrix_multiply(matrix, vector)
        vector = new_vector / np.linalg.norm(new_vector)

        # Estimate the eigenvalue with the Rayleigh quotient  $v^T M v$ .
        transpose_vector = transpose_matrix(vector)
        matrix_vector = matrix_multiply(matrix, vector)
        rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
        approximate_eigenvalue = rayleigh_quotient[0, 0]
        error = abs(approximate_eigenvalue - actual_eigenvalue)

        # Stop at the first iteration whose error is below 1e-7.
        if error < tolerance:
            return iteration

    # Return the limit to indicate that the requested accuracy was not reached.
    return 10000

iterations_for_lambda_1 = count_iterations(C, eigenvalues[0])
iterations_for_lambda_2 = count_iterations(deflated_matrix, eigenvalues[1])

print("Required accuracy:", 1e-7)
print("Iterations for lambda_1:", iterations_for_lambda_1)
print("Iterations for lambda_2:", iterations_for_lambda_2)
print("A larger gap between eigenvalues usually gives faster convergence.")
```

```
Required accuracy: 1e-07  
Iterations for lambda_1: 11  
Iterations for lambda_2: 6  
A larger gap between eigenvalues usually gives faster convergence.
```

In []: